

HTTP is "stateless," meaning it does not retain memory of previous requests. State is rebuilt through an identifier that the client resends with every request.

The browser retains only the cookie containing the session ID, while the server stores the session data. By default, Django stores sessions in the project's database, but this can be changed to store data elsewhere based on the project's requirements.

The client (web browser) sends a cookie containing the session ID with each new request, and Django's middleware intercepts this cookie on the server to retrieve the session data. On a successful login, Django stores the user's primary key in the session; on every later request, ``AuthenticationMiddleware`` reads it and populates ``request.user``.

The cookie backend allows clients to read session data with MAC protection, but poses a risk of cookie theft due to lack of invalidation upon logout.

A Session Fixation attack occurs when an attacker forces a user to use a session ID that the attacker already knows. Once the victim logs in, the attacker can use the same session ID to access the system as the user.

According to Mozilla (2025), the `SameSite` attribute helps prevent information leakage, preserves user privacy, and provides protection against cross-site request forgery (CSRF) attacks. Likewise, the `HttpOnly` attribute prevents JavaScript from accessing cookies, protecting them from XSS attacks. `Secure` forces the cookie to travel only through HTTPS.

Mozilla. (2025). *Using HTTP cookies - HTTP | MDN*. MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies>

To use MariaDB, you need to install a driver, either ``mysqlclient`` or ``MySQL Connector/Python``. You create the database and its user inside MariaDB with the `mysql` command-line client, granting that user the necessary privileges. Specifying the `utf8mb4` character set ensures all tables and columns use full UTF-8 by default. To connect, you update the ``DATABASES`` dictionary in your Django project's ``settings.py`` file to include the connection details: database name, user, password, host, and port.

``python manage.py migrate`` is the command in charge of applying or reversing schema migrations in your database. Also, the command ``mysqldump`` creates a backup of the data in a plain-text file, allowing the user to restore it if it is lost.

Django's migration system is designed to be independent of the database engine, so users do not need to rewrite migrations when switching databases.

MariaDB does not support transactional DDL, so a migration that fails midway cannot be rolled back automatically; the schema is left in a partial state and must be restored manually before retrying.

In contrast, SQLite supports transactional DDL, as in PostgreSQL, which means that if a migration fails, Django can automatically revert changes, keeping the database intact.

Running ``python manage.py sqlmigrate notes 0001`` on my own project printed the `CREATE TABLE "notes_note"` statement without touching the database. Django wrapped it in `BEGIN/COMMIT` and showed how each model field became a column: an auto-incrementing primary key, plus `NOT NULL` and length constraints derived from the model.

Django Software Foundation. (n.d.). *Databases*. Django Project.
<https://docs.djangoproject.com/en/6.0/ref/databases/>

Django Software Foundation. (n.d.). *Migrations*. Django Project.
<https://docs.djangoproject.com/en/6.0/topics/migrations/>